

DATA 260 Homework 3

Authentication and Retrieval Comparison

Grocery Supply and Recall Notices

This report documents the Homework 3 authentication system and the retrieval-only comparison of token, semantic, and sentence-window chunking. The authentication checks passed, the five-document FDA corpus exceeded the 200 KB requirement, and all three retrieval methods found the expected source within the top five results for every fixed question.

Section 0 Personal Configuration and Domain

Name - Vedant Vilas Vartak

SID4 - 3539

Port Base - 8839

Prefix - s3539

Seed - 3539

Verify Seed - 263539

Domain ID - 3

Assigned Domain - Grocery Supply and Recall Notices

Hardware - Apple M4 MacBook Air, 10 CPU cores, 16 GB memory

Local Embedding Model - sentence-transformers/all-MiniLM-L6-v2

Code and Results Commit - 268381dd37077b9c27084313e7dd264e1813322d

Final Submission Tag - hw3

GitHub Repository - <https://github.com/vedantvartakwork/data260-3539>

Part 1 FastAPI authentication and session management

I added a separate FastAPI authentication router with four routes. Starlette SessionMiddleware signs the session cookie, and the application also keeps a small server-side session registry so logout can revoke a session immediately. The idle timeout is 300 seconds.

Route	Behavior
GET /	Shows the domain home page and changes navigation based on login state.
GET and POST /login	Shows the login form, validates credentials, displays a Bootstrap alert for invalid credentials, and redirects valid users.
GET /dashboard	Displays the signed-in user and rejects missing, revoked, or expired sessions.
GET /logout	Removes the active server-side session, clears the signed cookie, and redirects home.

Route screenshots

Figure 1 Login page reached after a protected-route redirect

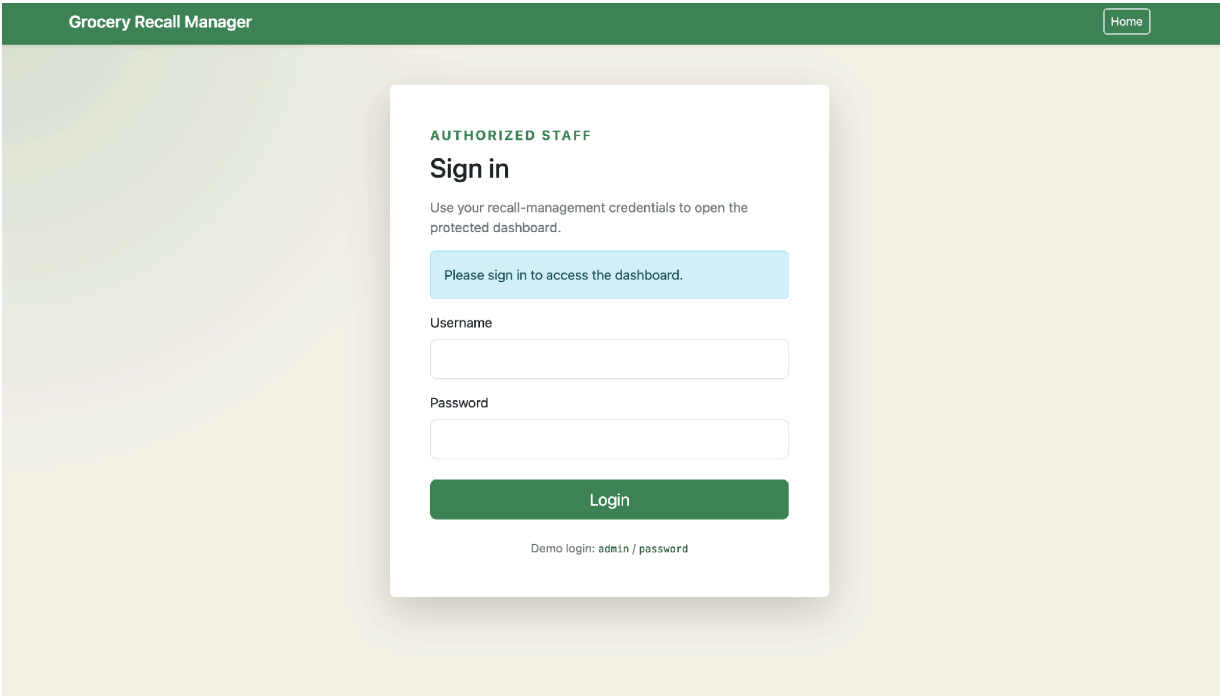


Figure 2 Protected dashboard after successful login

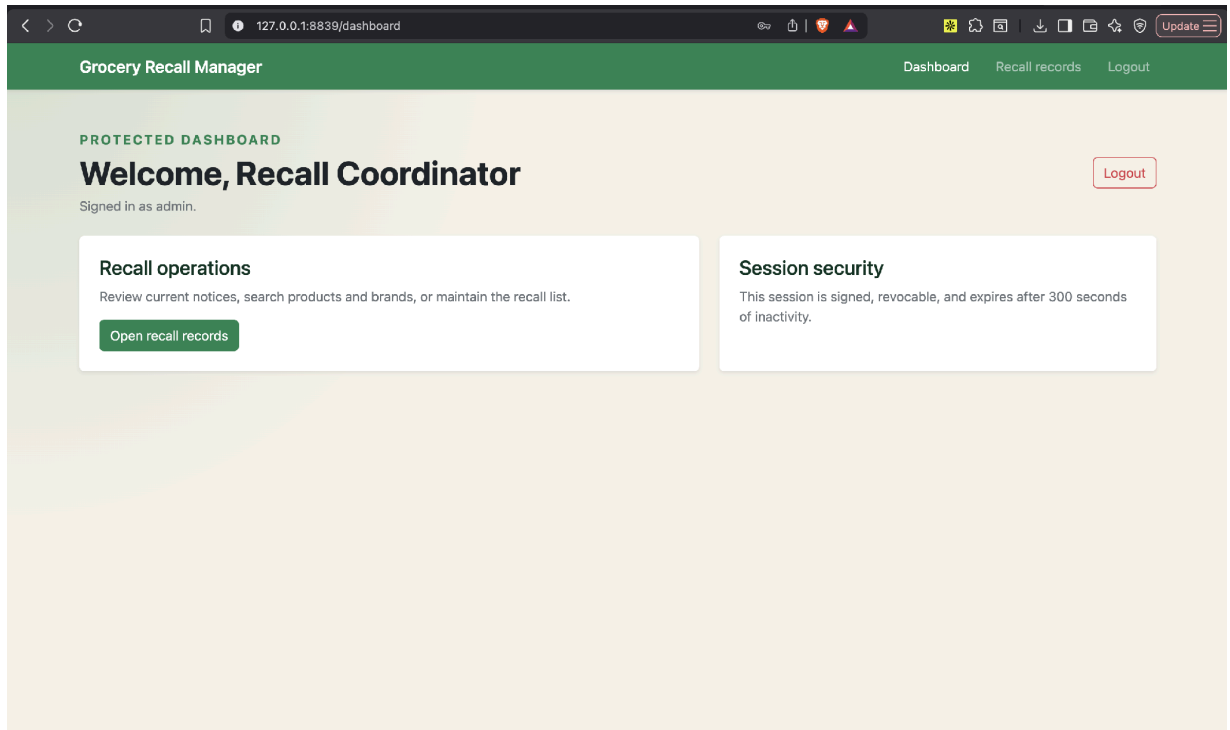


Figure 3 Home page while the user is logged in

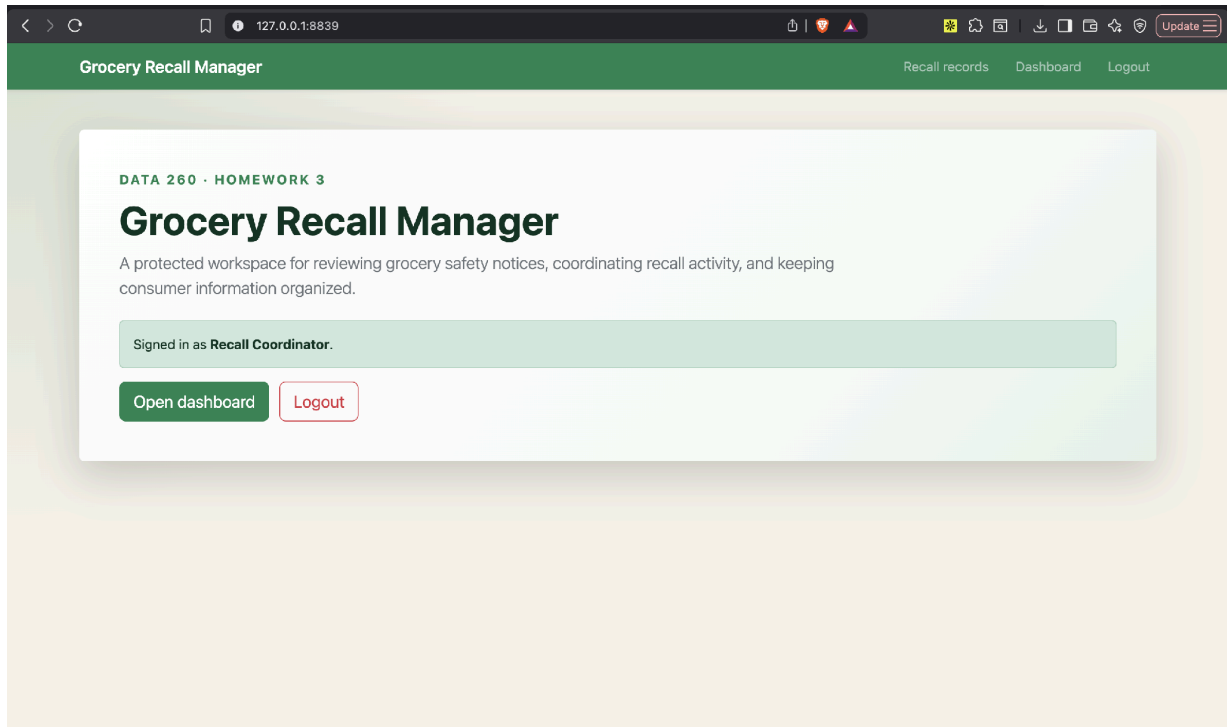


Figure 4 Home page after logout

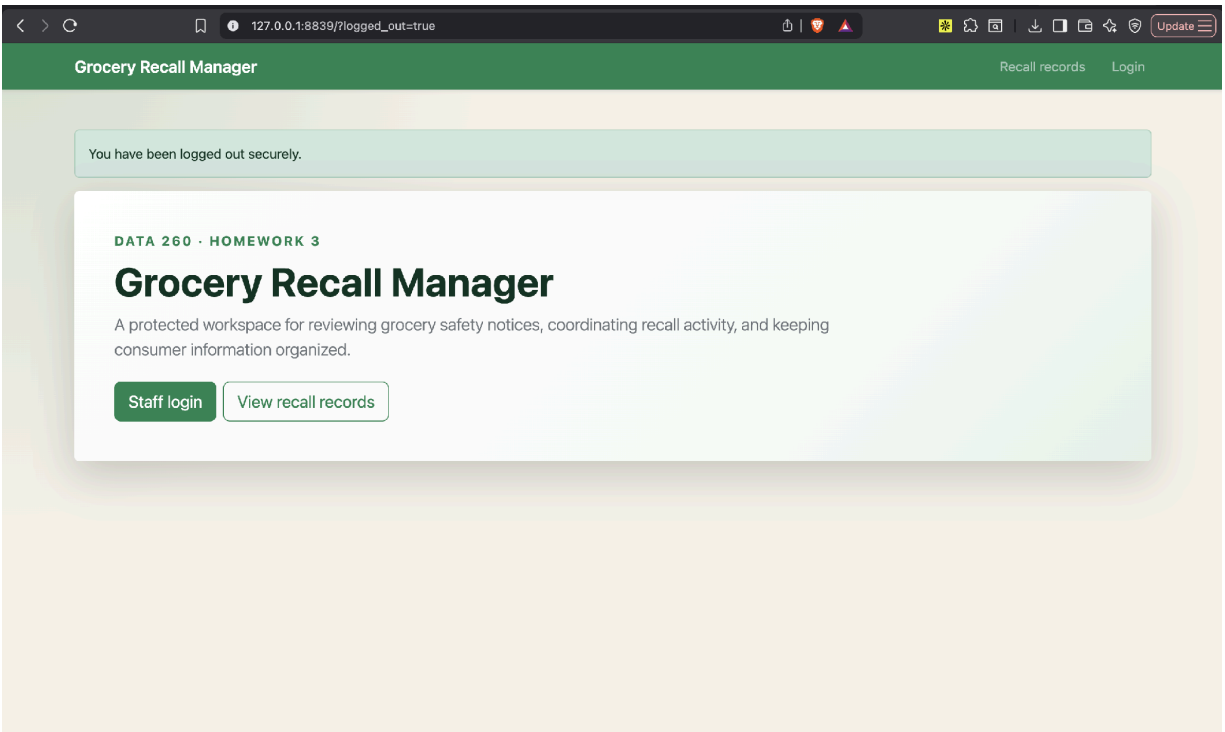
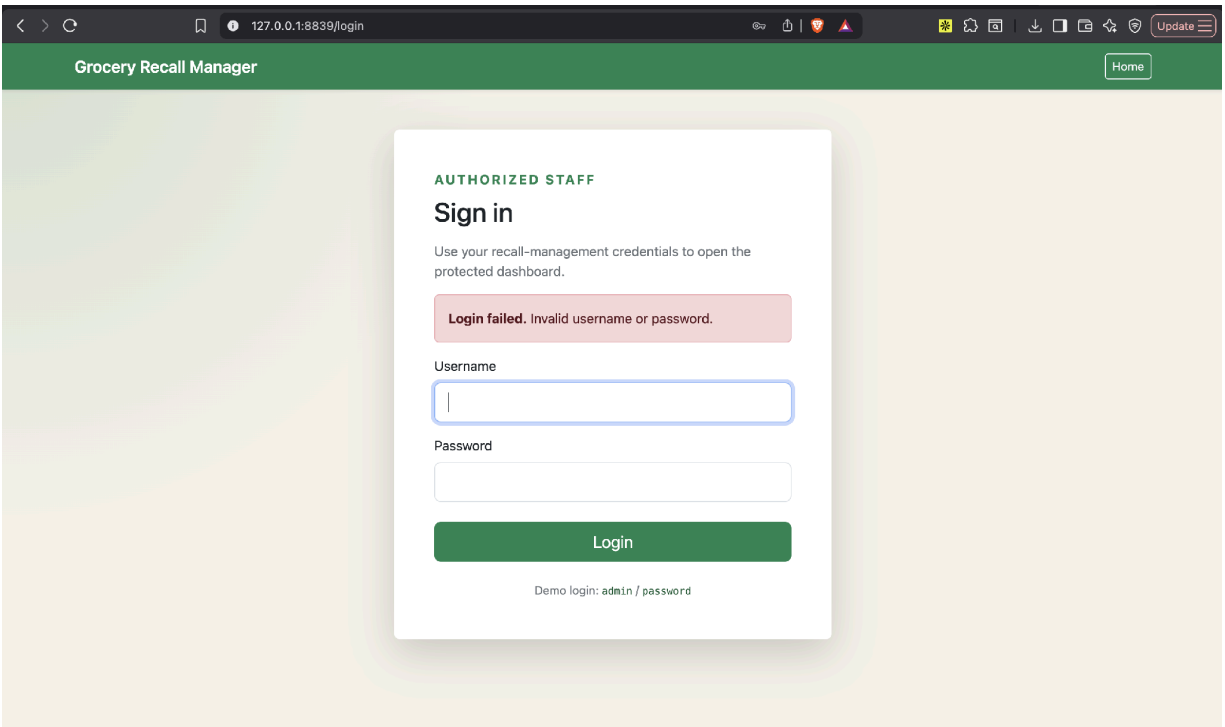


Figure 5 Bootstrap alert for invalid credentials



Cookie and session protection

The response header shows the HttpOnly, Secure, and SameSite=Lax attributes. The separate session check confirms that a logged-out cookie and an idle-expired session are both redirected to the login page instead of reaching the dashboard.

Figure 6 Secure Set-Cookie response header

```
$ .venv/bin/python scripts/show_hws_cookie_evidence.py
HOMEWORK 3 - SECURE SESSION COOKIE
POST /login status: 302
Redirect location: /dashboard
Set-Cookie response header:
set-cookie=eyJmYWRkbWVudDQ9ODc5MGMzZGZlOjE5YnVzZWJwZXI6OTY7bnVzZWJwZXI6ogfmbkbluiuwIeRpc3B8YXlfbmFtZSI6ICZSNHhhbGwgOwQ2PcmRpbmFob3JifSwleHRhc3RlYWNoeXZpdHkiOiAuehg5NAB0AYLjdXVjYNSdSxg.acq4xg.8Td4T8md83yTC0AM58VAMyvnyps; path=/; Max-Age=300; httponly; samesite=lax; secure
HttpOnly: PASS
Secure: PASS
SameSite=Lax: PASS

Saving session...
...copying shared history...
...saving history...truncating history files...
...completed.
Deleting expired sessions...none found.

[Process completed]
```

Figure 7 Logged-out and expired sessions rejected

```

C:\venv\bin\python scripts\show_hw3_session_evidence.py
HOMEWORK3 - 3 - SESSION REUSE PROTECTION
Login status: PASS - /dashboard
Logout status: 302 - /?logged-out=true
Replayed logged-out cookie: 302 - /?login?reason=session_expired
Result: PASS - the logged-out session cannot reach /dashboard

Expired idle session: 302 - /?login?reason=session_expired
Result: PASS - the expired session cannot reach /dashboard

Saving session...
...copying shared history...
...saving history...truncating history files...
...completed.

[Process completed]

```

Templates directory

Figure 8 Required Jinja templates directory

```
$ find code/web_application/templates -maxdepth 1 -type f -print | sort
code/web_application/templates/dashboard.html
code/web_application/templates/index.html
code/web_application/templates/login.html

Saving session...
...copying shared history...
...saving history...truncating history files...
...completed.

[Process completed]
```

Authentication router code

The assignment asks for only `auth.py` at the bottom of this section. The complete router is listed below.

```

"""Authentication routes for the grocery recall application."""

from __future__ import annotations

import os
import time

from pathlib import Path
from uuid import uuid4

from fastapi import APIRouter, Form, Request
from fastapi.responses import RedirectResponse
from fastapi.templating import Jinja2Templates
from starlette.status import HTTP_302_FOUND

router = APIRouter()

TEMPLATE_DIR = Path(__file__).resolve().parent.parent / "templates"
templates = Jinja2Templates(directory=TEMPLATE_DIR)

VALID_USERNAME = os.getenv("DEMO_USERNAME", "admin")
VALID_PASSWORD = os.getenv("DEMO_PASSWORD", "password")
IDLE_TIMEOUT_SECONDS = int(os.getenv("SESSION_IDLE_TIMEOUT_SECONDS", "300"))

# Starlette signs the browser cookie. This small server-side registry additionally
# lets logout revoke a session immediately, even if an old cookie is replayed.
ACTIVE_SESSIONS: dict[str, dict[str, object]] = {}

def reset_sessions() -> None:
    """Clear active sessions for deterministic tests and local demonstrations."""
    ACTIVE_SESSIONS.clear()

```

```

def _active_user(request: Request) -> tuple[dict[str, str] | None, str | None]:
    """Return the active user, enforcing revocation and an idle timeout."""
    session_id = request.session.get("session_id")
    user = request.session.get("user")
    record = ACTIVE_SESSIONS.get(session_id) if isinstance(session_id, str) else None

    if not isinstance(user, dict) or record is None:
        request.session.clear()
        return None, "login_required"

    now = time.time()
    last_activity = float(record.get("last_activity", 0.0))
    if now - last_activity > IDLE_TIMEOUT_SECONDS:
        ACTIVE_SESSIONS.pop(session_id, None)
        request.session.clear()
        return None, "session_expired"

    record["last_activity"] = now
    request.session["last_activity"] = now
    return user, None

@router.get("/")
def home(request: Request, logged_out: bool = False):
    """Show the domain home page with navigation based on login state."""
    user, _ = _active_user(request)
    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={"user": user, "logged_out": logged_out},
    )

@router.get("/login")
def login_page(request: Request, reason: str | None = None):
    """Show the login form and any session-status alert."""
    user, _ = _active_user(request)
    if user:
        return RedirectResponse("/dashboard", status_code=HTTP_302_FOUND)
    return templates.TemplateResponse(
        request=request,
        name="login.html",
        context={"error": None, "reason": reason},
    )

@router.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    """Validate credentials, create a revocable session, and redirect."""
    if username != VALID_USERNAME or password != VALID_PASSWORD:
        request.session.clear()
        return templates.TemplateResponse(
            request=request,
            name="login.html",
            context={"error": "Invalid username or password.", "reason": None},
            status_code=401,
        )

    request.session.clear()
    session_id = uuid4().hex
    user = {"username": username, "display_name": "Recall Coordinator"}
    now = time.time()
    ACTIVE_SESSIONS[session_id] = {"user": user, "last_activity": now}
    request.session.update({"session_id": session_id, "user": user, "last_activity": now})
    return RedirectResponse("/dashboard", status_code=HTTP_302_FOUND)

@router.get("/dashboard")
def dashboard(request: Request):
    """Show the protected dashboard only while the session remains active."""
    user, reason = _active_user(request)
    if not user:
        return RedirectResponse(f"/login?reason={reason}", status_code=HTTP_302_FOUND)
    return templates.TemplateResponse(
        request=request,
        name="dashboard.html",
        context={"user": user, "idle_timeout": IDLE_TIMEOUT_SECONDS},
    )

@router.get("/logout")
def logout(request: Request):
    """Revoke the server-side session, clear its cookie, and return home."""
    session_id = request.session.get("session_id")
    if isinstance(session_id, str):
        ACTIVE_SESSIONS.pop(session_id, None)

```

```
request.session.clear()
return RedirectResponse("/?logged_out=true", status_code=HTTP_302_FOUND)
```

Part 2 LlamaIndex chunking comparison

I compared TokenTextSplitter, SemanticSplitterNodeParser, and SentenceWindowNodeParser using the same five questions, the same public embedding model, and top-k equal to 5. Each method built its own in-memory VectorStoreIndex with SimpleVectorStore. Tiny Shakespeare was used only as a warm-up; the graded results use the FDA corpus described below.

Dataset and fixed questions

The graded corpus contains five local FDA document snapshots totaling 501,880 bytes of extracted text. SOURCES.md records the URLs and access date, while CORPUS_MANIFEST.json records filenames, byte sizes, and SHA-256 hashes. The five questions were committed before the retrieval outputs were added.

Graded text source	Bytes
FDA Regulatory Procedures Manual Chapter 7	336,558
FDA 2026 Investigations Operations Manual Chapter 7	68,400
Retail Consignee Lists Guidance	19,013
Public Warning Guidance	30,241
Reportable Food Registry Questions and Answers	47,668

ID	Question focus	Expected source	Source unique
q1	Reportable-food reporting deadline	RFR guidance	Yes
q2	When retail consignee lists are published	Consignee guidance	Yes
q3	Information required in a public warning	Public warning guidance	Yes
q4	Minimum recall-strategy items	IOM Chapter 7	No
q5	Common effectiveness-check elements	RPM Chapter 7	No

Chunking and indexing code

The three parsers use a 256-token chunk with 40-token overlap, semantic boundaries with a one-sentence buffer, and a two-sentence window around each sentence.

```
def build_nodes(documents, embed_model):
    chunkers = {
        "token": TokenTextSplitter(chunk_size=256, chunk_overlap=40),
        "semantic": SemanticSplitterNodeParser(
            embed_model=embed_model,
            buffer_size=1,
            breakpoint_percentile_threshold=95,
        ),
        "sentence_window": SentenceWindowNodeParser.from_defaults(
            window_size=2,
```

```

        window_metadata_key="window",
        original_text_metadata_key="original_text",
    ),
}
return {
    name: chunker.get_nodes_from_documents(documents, show_progress=False)
    for name, chunker in chunkers.items()
}

def build_index(nodes, embed_model):
    storage_context = StorageContext.from_defaults(
        vector_store=SimpleVectorStore()
    )
    return VectorStoreIndex(
        nodes=nodes,
        embed_model=embed_model,
        storage_context=storage_context,
        show_progress=False,
    )

```

Retrieval and raw output code

For every question, the helper computes the query embedding, times only the similarity search, explicitly embeds each returned chunk, calculates cosine similarity, and saves the required fields.

```

query_vector = np.asarray(
    embed_model.get_query_embedding(query), dtype=np.float32
)
retriever = index.as_retriever(similarity_top_k=min(top_k, len(nodes)))
started = perf_counter()
retrieved = retriever.retrieve(query)
latency_ms = (perf_counter() - started) * 1000

texts = [chunk.text(item.node) for item in retrieved]
doc_vectors = np.asarray(
    embed_model.get_text_embedding_batch(texts), dtype=np.float32
)

rows = []
for rank, (item, text, vector) in enumerate(
    zip(retrieved, texts, doc_vectors, strict=True), start=1
):
    source_file = str(item.node.metadata.get("file_name", "unknown"))
    rows.append({
        "rank": rank,
        "store_score": float(item.score),
        "cosine_sim": cosine_similarity(query_vector, vector),
        "chunk_len": len(text),
        "source_file": source_file,
        "preview": " ".join(text.split()[:160]).rstrip(),
    })

```

The raw directory contains 15 individual JSON files, one for each question and technique, plus combined CSV and JSONL files. The metrics script reads the JSONL file and recomputes the table instead of using hand-entered numbers.

Figure 9 Shared-query embeddings and top-five retrieval output for all three techniques

```
$ .venv/bin/python scripts/show_hw3_retrieval_evidence.py
HOMEWORK 3 - SHARED QUERY RETRIEVAL EVIDENCE

TECHNIQUE: token
Query: What information should a public food recall warning give consumers so they can identify the product and understand the risk?
Query embedding: dimension=384, first8=[0.011959, 0.0055616, -0.039686, 0.018528, 0.045662, 0.10124, 0.038579, 0.0055561]
Vector shapes: query=(384,), documents=(5, 384)
Retrieval latency: 9.715 ms
rank  store_score  cosine_sim  chunk_len  source  preview
1      0.8011      0.8011      1280      fda_public_warning_guidance.txt  the product is limited to a small number of consignees that are easily identified and can
2      0.7920      0.7920      1170      fda_retail_consignee_lists_guidance.txt  information otherwise exempt from public disclosure, such as CCI, is nevertheless availabl
3      0.7647      0.7647      975       fda_public_warning_guidance.txt  own public warning. Additionally, the FDA may publicly issue information that may address
4      0.7639      0.7639      1192      fda_retail_consignee_lists_guidance.txt  FDA Food Safety Modernization Act (FSMA), which directs the Agency, in conducting recalls
5      0.7572      0.7572      1162      fda_retail_consignee_lists_guidance.txt  may be able to seek medical treatment to prevent illness after exposure to a contaminated

TECHNIQUE: semantic
Query: What information should a public food recall warning give consumers so they can identify the product and understand the risk?
Query embedding: dimension=384, first8=[0.011959, 0.0055616, -0.039686, 0.018528, 0.045662, 0.10124, 0.038579, 0.0055561]
Vector shapes: query=(384,), documents=(5, 384)
Retrieval latency: 7.043 ms
rank  store_score  cosine_sim  chunk_len  source  preview
1      0.7318      0.7318      12897      fda_retail_consignee_lists_guidance.txt  == Page 5 == Contains Nonbinding Recommendations 2 II. BACKGROUND This guidance outlines
2      0.7216      0.7216      2867      fda_retail_consignee_lists_guidance.txt  To discuss an alternative approach, contact the FDA staff responsible for this guidance as
3      0.7050      0.7050      13263      fda_public_warning_guidance.txt  III. Discussion: The FDA's policy is to evaluate the particular circumstances of each indi
4      0.7010      0.7010      3053      fda_public_warning_guidance.txt  9 This webpage also provides links to various FDA Center webpages, where public warnings o
5      0.6939      0.6939      12045      fda_regulatory_procedures_manual_chapter_7.txt  4. Notification to United States Department of Agriculture (USDA) 4.1 Division office resp

TECHNIQUE: sentence_window
Query: What information should a public food recall warning give consumers so they can identify the product and understand the risk?
Query embedding: dimension=384, first8=[0.011959, 0.0055616, -0.039686, 0.018528, 0.045662, 0.10124, 0.038579, 0.0055561]
Vector shapes: query=(384,), documents=(5, 384)
Retrieval latency: 28.313 ms
rank  store_score  cosine_sim  chunk_len  source  preview
1      0.7948      0.7948      506      fda_public_warning_guidance.txt  As such, a public warning should include: a) information to help identify the recalled pro
2      0.7640      0.7640      420      fda_public_warning_guidance.txt  The following recalls generally present examples of serious hazards to health such that a
3      0.7617      0.7617      226      fda_retail_consignee_lists_guidance.txt  BACKGROUND This guidance outlines the circumstances under which FDA intends to collect, co
4      0.7607      0.7607      406      fda_retail_consignee_lists_guidance.txt  Specifically, FDA intends to publicize retail consignee lists for such food recalls when b
5      0.7547      0.7547      513      fda_retail_consignee_lists_guidance.txt  I. INTRODUCTION This guidance provides information on how and when FDA intends to publiciz

Saving session...
...copying shared history...
...saving history...truncating history files...
...completed.

[Process completed]
```

Retrieval quality results

Technique	Chunks	Avg length	Top 1	Mean at k	Recall at k	Latency ms
Token	540	1108.32	0.7652	0.7266	100.0%	11.182
Semantic	154	3246.10	0.7151	0.6773	100.0%	7.479
Sentence window	2934	170.38	0.8214	0.7927	100.0%	31.457

Recall@k is the fraction of questions for which at least one of the top five chunks came from the expected source. Every method reached 100 percent expected-source recall for these five questions.

Confidently scored wrong retrieval

For q3, token retrieval ranked a chunk from the retail consignee guidance second with cosine similarity 0.7920, even though the expected answer is in the public warning guidance. The incorrect chunk discusses public disclosure, recalls, and health protection, so its vocabulary is very close to the question. It does not contain the source-unique list of product-identification and risk details, which shows why a high embedding score does not guarantee that a chunk answers the question.

Figure 10 Verification, comparison metrics, and confidently scored wrong result

```
$ make verify-hw03
Verification: PASS - 24 tests, 5 corpus files (501880 bytes), 15/15 retrieval result files

$ sed -n '1,80p' reports/hw03/METRICS.md
# Homework 3 Retrieval Metrics

Model: 'sentence-transformers/all-MiniLM-L6-v2'; top-k: '5'; questions: '5'.

| Technique | Chunks | Avg chunk length | Top-1 cosine | Mean@k cosine | Recall@k | Mean retrieval latency (ms) |
| --- | --- | --- | --- | --- | --- | --- |
| Semantic | 154 | 3246.10 | 0.7151 | 0.6773 | 100.0% | 7.479 |
| Sentence window | 2934 | 170.38 | 0.8214 | 0.7927 | 100.0% | 31.457 |
| Token | 540 | 1108.32 | 0.7652 | 0.7266 | 100.0% | 11.182 |

Recall@k is the share of questions where at least one of the top-k chunks came from the expected source file.

## Per-question expected-source recall

| Question | Token | Semantic | Sentence window |
| --- | --- | --- | --- |
| q1 | Yes | Yes | Yes |
| q2 | Yes | Yes | Yes |
| q3 | Yes | Yes | Yes |
| q4 | Yes | Yes | Yes |
| q5 | Yes | Yes | Yes |

## Confidently scored wrong retrieval

- Question: q3 - What information should a public food recall warning give consumers so they can identify the product and understand the risk?
- Technique and rank: Token rank 2
- Cosine similarity: 0.7920
- Expected source: 'fda_public_warning_guidance.txt'
- Retrieved source: 'fda_retail consignee_lists_guidance.txt'
- Preview: information otherwise exempt from public disclosure, such as CCI, is nevertheless available for public disclosure to the extent necessary to effectuate a
- Why it likely scored well: both passages use closely related FDA recall, public-health, and notification vocabulary even though this source does not contain the sou
wer.

Saving session...
...copying shared history...
```

Observations

Sentence-window chunking produced the strongest similarity results. Its average top-1 cosine was 0.8214 and its mean@5 cosine was 0.7927, both higher than token and semantic chunking. The short sentence-sized chunks focused the embedding on one statement at a time, and the attached window metadata kept nearby context available. The cost was a larger index of 2,934 chunks and the slowest mean retrieval latency at 31.457 ms.

Semantic chunking created only 154 chunks and had the fastest mean retrieval latency at 7.479 ms, but its chunks averaged 3,246 characters. Those long passages often mixed relevant and unrelated material, which likely reduced both its top-1 and mean@5 cosine scores. Token chunking was the middle option: it had 540 chunks, 11.182 ms mean latency, and similarity scores between the other two methods.

Conclusion

I judge sentence-window chunking to be the best choice for this corpus because it produced the highest top-1 and mean@5 cosine scores while keeping 100 percent recall@5. Token chunking is a reasonable latency-quality compromise, but its similarity scores were lower. Semantic chunking was fastest, although its large chunks were less precise for these questions.

Reproducibility and verification

- make run-web starts the application on port 8839.
- make warmup-hw3 runs the Tiny Shakespeare warm-up only.
- make experiment-hw3 reruns all five questions with all three chunkers and writes raw output plus RUN_LOG.txt.
- make metrics-hw3 rebuilds METRICS.md from the saved JSONL output.
- make verify-hw03 runs the tests and checks the corpus, manifest, questions, raw output, log, and metrics.

The final self-check passed 24 tests, verified five corpus files totaling 501,880 bytes, confirmed every manifest hash, and found all 15 per-query retrieval files.

Use of an assistant

What I used assistance for and what I did myself

I used an assistant to help organize the assignment requirements, draft the first version of the authentication and retrieval scripts, and review the repository structure. I checked the assignment again, ran the application and experiments, verified the outputs, captured the screenshots, and reviewed the final code and report.

One unsuitable output or independently verified item

One draft of the metrics helper treated the first retriever row as the top-1 cosine value. That was not precise enough for this assignment because the instructions define top-1 as the highest explicit cosine similarity among the returned top-k chunks.

How I detected or verified it

I found the issue by rereading the metric definition in the homework PDF and comparing it with the saved per-query cosine values. I also recomputed the table directly from the JSONL file and checked the result against the terminal output.

What I changed and why it works

I changed the summary calculation to take the maximum explicit cosine value from each question's top-k results before averaging across the five questions. This matches the definition in the assignment and keeps the reported metrics reproducible from the raw files.